COMPARATIVE ANALYSIS REPORT

A Step-by-Step & Feature-by-Feature Comparison of Movie Recommendation System Notebooks

Target Notebook 1: Movie Recommendation System.ipynb	**Target Notebook 2:** Netflix_Movie_Recommendation_System.ipynb
Scope: 3 Datasets (Netflix, IMDb Movies, Real Ratings)	**Scope:** 1 Dataset (Netflix) + Synthetic Ratings Simulation
Pipeline Depth: 28-Step Workflow (Real CF Engine)	**Pipeline Depth:** 28-Step Workflow (Synthetic CF Simulation)

1. Executive Summary & Core Architectural Differences

This comparative analysis presents an exhaustive evaluation of two data science pipelines developed for Movie Recommendation Systems: **`Movie Recommendation System.ipynb`** (referred to as *Notebook 1* or *NB1*) and **`Netflix_Movie_Recommendation_System.ipynb`** (referred to as *Notebook 2* or *NB2*). While both notebooks adhere to an identical 28-step structural blueprint—spanning data ingestion, EDA, cleaning, outlier handling, feature engineering, encoding, scaling, and dashboarding—they diverge significantly in their data architecture, domain scope, mathematical formulation, and recommendation modeling strategy.

Architectural Dimension	Movie Recommendation System (NB1)	Netflix Movie Recommendation (NB2)
Dataset Architecture	**Multi-Dataset Hybrid**: Integrates 3 distinct CSVs: • netflix_data.csv (8,807 x 12) • movies.csv IMDb (9,999 x 9) • ratings.csv Real Logs (97,157 x 4)	**Single-Dataset Focus**: Operates on 1 CSV: • netflix_data.csv (8,807 x 12) • Generates synthetic user interaction logs in-memory (5,000 ratings)
Collaborative Filtering Data	**Empirical Real Ratings**: Uses real user-item interaction log (ratings.csv) with 97,157 ratings across 610 users and 9,724 movies.	**Synthetic Rating Simulation**: Generates 5,000 synthetic ratings using np.random across 200 virtual user IDs mapped to Netflix show_ids.
Text Vectorization Strategy	**Dual Vectorizer Approach**: Fits both TF-IDF Matrix (8807 x 5000) and CountVectorizer Matrix (8807 x 2000) with n-grams (1,2).	**Single TF-IDF Vectorizer**: Fits TF-IDF Matrix (8807 x 5000) and performs token vocabulary inspection (get_feature_names_out).
Outlier Treatment Strategy	**Analytical Capping (Winsorization)**: Implements np.clip() capping on Netflix duration and IMDb release year/runtime.	**Filtering & Metric Table**: Quantifies duration bounds and filters extreme values while constructing detailed IQR vs Z-Score metric tables.
Correlation & Multivariate Scope	Analyzes Pearson correlation across IMDb ratings, votes, and runtime. Dashboard presents a 12-panel 3x4 layout.	Analyzes Pearson correlation across Netflix temporal features (release year, added year, duration). Dashboard presents a 6-panel 2x3 layout.
Step 28 Comparison Matrix	Executed Pandas DataFrame printed directly as raw text output.	Formatted Markdown table rendered natively within a notebook cell.

2. Step-by-Step Comparative Analysis (Steps 1 to 28)

The table below provides a granular step-by-step contrast across all 28 workflow steps, highlighting operational implementations, data structures, and analytical implications.

Step # & Name	Notebook 1: Movie Recommendation System.ipynb	Notebook 2: Netflix_Movie_Recommendation_System.ipynb	Key Difference & Impact
Step 1: Import Libraries	Imports Pandas, NumPy, Matplotlib, Seaborn, SciPy, TFIDF, CountVectorizer, LabelEncoder, StandardScaler, MinMaxScaler, Cosine Sim.	Imports standard stack plus explicit OneHotEncoder module. Uses Seaborn darkgrid/whitegrid styling.	NB1 includes CountVectorizer for text n-grams; NB2 focuses on TF-IDF text features.
Step 2: Load Dataset	Loads 3 CSV files: netflix_data.csv (8,807x12), movies.csv (9,999x9), ratings.csv (97,157x4). Prints shapes and samples.	Loads 1 CSV file: netflix_data.csv (8,807x12). Uses IPython display() for head, tail, sample, and column lists.	NB1 establishes a hybrid recommendation infrastructure; NB2 focuses strictly on Netflix content catalog.
Step 3: Understand Dataset	Calls .info(), .dtypes, .nunique(), and value_counts() on all 3 loaded DataFrames.	Focuses metadata inspection exclusively on netflix_data.csv, printing column types and unique counts.	NB1 examines cross-dataset structures; NB2 performs deep single-table schema verification.
Step 4: Initial EDA	Analyzes distribution of content type (Movie vs TV Show) across Netflix and IMDb movies.	Plots content type breakdown with custom Netflix brand color scheme (#e50914 red and #221f1f black).	NB2 incorporates domain-specific color styling for Netflix visual identity.
Step 5: Missing Value Analysis	Computes total missing values and percentage across netflix_df, movies_df, and ratings_df.	Analyzes missing values in netflix_data.csv and generates a visual bar chart of missing counts per column.	NB2 provides visual missing value diagnostic plots; NB1 provides text tabular outputs across 3 tables.
Step 6: Duplicate Analysis	Executes .duplicated().sum() check across netflix_df, movies_df, and ratings_df.	Executes duplicate check on netflix_data.csv title and show_id columns.	Identical duplicate-free status confirmed for netflix_data.csv in both pipelines.
Step 7: Data Type Correction	Converts date_added to datetime and extracts duration_num for Netflix and runtime_num for IMDb movies.	Converts date_added to datetime, extracts added_year, and parses duration into duration_min and duration_num.	NB2 creates explicit added_year and duration_min columns for feature engineering.
Step 8: Basic Data Cleaning	Fills missing director, cast, country with 'Unknown' and rating with 'TV-MA' across netflix_df.	Fills missing director, cast, country with 'Unknown', rating with mode, and missing duration with median.	NB2 applies statistical mode and median imputation for numeric and categorical missing values.
Step 9: Univariate EDA	Plots univariate distributions for Netflix content type, IMDb ratings, and rating counts.	Plots univariate distribution focusing on Netflix duration_min, release_year, and top maturity ratings.	NB1 covers multi-dataset univariate distributions; NB2 focuses on Netflix catalog distributions.
Step 10: Bivariate EDA	Analyzes Bivariate relationships: Content Type vs Rating, IMDb Rating vs Runtime.	Analyzes Content Type vs Duration, and Added Year vs Content Type over time.	NB2 investigates temporal trends of Netflix additions over time.
Step 11: Multivariate EDA	Plots Content Type x Rating x Duration across datasets.	Plots Genre x Content Type x Release Year relationship.	NB2 focuses on multi-genre distribution across release eras.
Step 12: Outlier Analysis (IQR)	Calculates Q1, Q3, IQR for duration_num and release_year across netflix_df and movies_df.	Calculates Q1, Q3, IQR specifically for movie duration_min in netflix_data.csv.	NB1 evaluates outliers across multiple datasets; NB2 concentrates on catalog duration outliers.

Step # & Name	Notebook 1: Movie Recommendation System.ipynb	Notebook 2: Netflix_Movie_Recommendation_System.ipynb	Key Difference & Impact
Step 13: Outlier Analysis (Z-Score)	Applies scipy.stats.zscore (|Z| > 3) on duration_num and release_year across datasets.	Applies scipy.stats.zscore (|Z| > 3) on movie duration_min and prints mean/std metrics.	Identical mathematical Z-score threshold (|Z| > 3) applied with different target scopes.
Step 14: Compare IQR & Z-Score	Constructs comparison DataFrame listing outlier counts for Duration and Release Year.	Constructs comprehensive metric table listing Method Criteria, Lower/Upper Bounds, Counts, and %.	NB2 delivers higher statistical reporting granularity (bounds, percentages, exact formulas).
Step 15: Outlier Capping / Filter	Performs Winsorization capping using np.clip(lower, upper) creating duration_capped.	Filters extreme movie duration outliers based on IQR bounds and plots before/after boxplots.	NB1 preserves sample size via capping; NB2 demonstrates filtering/visual truncation.
Step 16: Deep EDA - Movie Data	Executes 4-panel subplot analyzing release year trends, top genres, and IMDb gross revenue.	Executes deep movie metadata EDA focusing on top 10 primary genres and release era trends.	NB1 incorporates financial/IMDb metrics; NB2 stays strictly catalog-focused.
Step 17: Deep EDA - Rating Data	Analyzes real user interaction log from ratings.csv (97,157 ratings, user score distribution).	Generates 5,000 synthetic ratings using np.random for 200 synthetic users across show_ids.	CRITICAL DIVERGENCE: NB1 uses real user behavioral data; NB2 simulates collaborative ratings.
Step 18: GroupBy Analysis	Groups by type and listed_in computing mean, std, min, max duration across netflix_df.	Groups by type and rating computing mean, median, count, min, max using observed=False.	NB2 includes median statistic and observed=False parameter for categorical grouping.
Step 19: Aggregation	Applies named aggregation tuples (Total_Items, Mean_Duration, Min_Year, Max_Year).	Applies dictionary-based multi-column aggregation ({release_year: ['min','max'], duration_num: ['mean']}).	NB2 uses nested column multi-index aggregation outputs.
Step 20: Correlation Analysis	Computes Pearson correlation matrix on IMDb movies.csv (rating_num, votes_num, runtime_num).	Computes Pearson correlation matrix on Netflix features (release_year, added_year, duration_num).	NB1 measures user popularity correlations; NB2 measures temporal feature correlations.
Step 21: Content-Based Features	Concatenates title, director, cast, listed_in, description; fits TF-IDF Vectorizer.	Cleans text strings, creates combined_features column, fits TF-IDF Vectorizer (8807x5000).	Both build TF-IDF matrices; NB2 adds explicit string cleaning transformations.
Step 22: Collaborative Filtering	Filters top 200 users and top 200 movies from real ratings.csv; creates 200x200 dense pivot table.	Creates pivot table user_movie_matrix (user_id x show_id) from 5,000 synthetic ratings log.	NB1 yields high-density real user matrix; NB2 yields sparse simulated interaction matrix.
Step 23: Encoding	Applies LabelEncoder on type & rating; get_dummies One-Hot Encoding on type.	Applies pd.get_dummies One-Hot Encoding on type; LabelEncoder on rating.	Identical encoding logic structured with slightly different variable names.
Step 24: Text Feature Engineering	Fits CountVectorizer(stop_words='english', max_features=2000, ngram_range=(1,2)).	Inspects TF-IDF vocabulary using get_feature_names_out() and prints sample token terms.	NB1 creates n-gram word count matrix; NB2 verifies TF-IDF term tokenization.
Step 25: Scaling / Normalization	Applies StandardScaler and MinMaxScaler on release_year and duration_capped (fills NaN with 0).	Fills NaN with median(), applies StandardScaler and MinMaxScaler, returns explicit DataFrames.	NB2 uses median imputation prior to scaling and retains DataFrame column names.
Step 26: Final Feature Dataset	Prints 6-point integrity checklist (DataFrame shapes, matrix shapes, NaN/duplicate counts).	Prints 5-point feature verification check confirming matrix readiness for modeling.	Both establish rigorous pre-modeling data integrity verification.

Step # & Name	Notebook 1: Movie Recommendation System.ipynb	Notebook 2: Netflix_Movie_Recommendation_System.ipynb	Key Difference & Impact
Step 27: Pre-Modeling Visuals	Generates massive 12-panel dashboard grid (3x4 subplots) covering all EDA metrics.	Generates clean 6-panel visual grid (2x3 subplots) focusing on primary Netflix metrics.	NB1 provides exhaustive 12-chart visual summary; NB2 provides streamlined 6-chart summary.
Step 28: Feature Matrix Compare	Prints Pandas DataFrame comparison matrix contrasting Content-Based vs Collaborative Filtering.	Renders formatted Markdown comparison table inside a notebook markdown cell.	NB2 delivers superior notebook readability via rendered Markdown table formatting.

3. Deep Feature & Methodology Comparison Matrix

To evaluate the technical rigor of both pipelines, the table below compares key data science techniques, vectorization algorithms, scaling formulas, and matrix dimensions.

Feature Category	Methodology / Metric	Notebook 1 (Movie Recommendation)	Notebook 2 (Netflix Recommendation)
Text Vectorization	TF-IDF Vectorizer	Matrix Shape: (8807, 5000) stop_words='english'	Matrix Shape: (8807, 5000) stop_words='english', clean text
Text Vectorization	Count Vectorizer	Matrix Shape: (8807, 2000) ngram_range=(1,2)	Not implemented (TF-IDF term inspection used instead)
Collaborative Filtering	Interaction Matrix	Pivot Shape: (200, 200) Dense subset from 97,157 real ratings	Pivot Shape: (200, N) Sparse matrix from 5,000 synthetic ratings
Outlier Handling	IQR Method	Evaluated on Netflix Duration & IMDb Release Year	Evaluated on Netflix Duration with exact bounds table
Outlier Handling	Z-Score Method	scipy.stats.zscore (|Z| > 3)	scipy.stats.zscore (|Z| > 3)
Outlier Treatment	Capping / Filtering	np.clip() Winsorization (duration_capped)	IQR boundary truncation and filtering
Feature Scaling	Standard & MinMax	StandardScaler & MinMaxScaler (NaN filled with 0)	StandardScaler & MinMaxScaler (NaN filled with median)
Categorical Encoding	One-Hot & Label	LabelEncoder (type, rating) get_dummies (type)	pd.get_dummies (type) LabelEncoder (rating)
Dashboard Visuals	Pre-Modeling Dashboard	12 subplots (3x4 grid layout)	6 subplots (2x3 grid layout)

4. Technical Evaluation: Strengths & Limitations

Notebook 1: Movie Recommendation System.ipynb

Strengths:

- **Real-World Collaborative Data:** Leverages 97,157 real user ratings from ratings.csv, allowing authentic collaborative filtering models (SVD, Cosine Sim, User-User CF).
- **Hybrid Multi-Dataset Synergy:** Combines catalog metadata with IMDb movie ratings and user logs, offering richer feature interactions.

- **Exhaustive Visual Analytics:** 12-panel dashboard provides complete visual diagnostics before model execution.
- **Dual Text Representations:** Computes both TF-IDF embeddings and CountVectorizer n-gram counts.

Limitations:

- Imputes missing numeric values with 0 prior to scaling, which can distort standard deviation and Z-score distributions.
- Step 28 outputs the final comparison table as raw printed text rather than formatted Markdown.

Notebook 2: Netflix_Movie_Recommendation_System.ipynb

Strengths:

- **Clean Single-Dataset Focus:** Streamlined execution focused exclusively on Netflix catalog content.
- **Robust Imputation:** Imputes missing numerical values using column medians, preserving distribution shape prior to scaling.
- **Self-Contained Rating Simulation:** Enables collaborative filtering code validation even when external user rating files are unavailable.
- **Superior Notebook Formatting:** Utilizes rendered Markdown tables, custom visual brand colors (#e50914), and IPython display() calls.

Limitations:

- Synthetic user ratings (generated via np.random) lack true user preference patterns, limiting collaborative filtering accuracy.
- Lacks external IMDb ratings/revenue metadata for multi-catalog recommendation modeling.

5. Final Conclusion & Recommendations

Summary of Findings:
Both notebooks demonstrate high technical competency in executing a structured 28-step data science pipeline for movie recommendation systems. However, they serve different analytical objectives:
1. **`Movie Recommendation System.ipynb` (NB1)** represents an *empirical, production-grade hybrid recommendation pipeline*. By leveraging 97,157 real user ratings alongside Netflix and IMDb catalogs, NB1 delivers genuine collaborative filtering and hybrid recommendation capabilities.
2. **`Netflix_Movie_Recommendation_System.ipynb` (NB2)** represents a *clean, self-contained educational catalog pipeline*. It excels in data cleaning rigor (median imputation, explicit scaling DataFrames) and visual presentation, relying on synthetic ratings to demonstrate collaborative filtering mechanics without external dependencies.

Final Recommendation for Model Deployment:
For real-world recommendation deployment, **Notebook 1 (NB1)** is superior due to its real user interaction data. However, integrating NB2's data cleaning practices (median imputation and Markdown formatting) into NB1's real-data architecture would yield the ultimate hybrid recommendation system.